

# Panduan Implementasi Aplikasi Login User Menggunakan MVVM + Room Database

---

## 1. Tujuan Latihan

Pada praktikum ini mahasiswa akan membuat aplikasi login sederhana menggunakan:

- Kotlin
- Jetpack Compose
- MVVM Architecture
- Room Database
- SQLite

Fitur aplikasi:

- Input username
  - Input password
  - Login validation
  - Menampilkan status berhasil/gagal
  - Penyimpanan data menggunakan Room Database
- 

## 2. Software yang Dibutuhkan

### A. Android Studio

Gunakan:

- Android Studio Hedgehog / Iguana / versi terbaru

### B. Minimum SDK

Gunakan:

Minimum SDK : API 24

### C. Bahasa Pemrograman

Kotlin

---

### 3. Membuat Project Baru

#### Langkah 1

Buka Android Studio.

#### Langkah 2

Klik:

New Project

#### Langkah 3

Pilih:

Empty Activity

#### Langkah 4

Isi konfigurasi:

| Bagian       | Isi                   |
|--------------|-----------------------|
| Name         | LoginMVVMAApp         |
| Package Name | com.example.loginmvvm |
| Language     | Kotlin                |
| Minimum SDK  | API 24                |

Klik:

Finish

### 4. Menambahkan Dependency

Buka file:

build.gradle (Module: app)

Tambahkan dependency berikut:

```
implementation("androidx.room:room-runtime:2.6.1")
implementation("androidx.room:room-ktx:2.6.1")
kapt("androidx.room:room-compiler:2.6.1")

implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.8.0")
implementation("androidx.navigation:navigation-compose:2.7.7")
```

## 5. Mengaktifkan KAPT

Tambahkan plugin berikut:

```
plugins {  
    id("kotlin-kapt")  
}
```

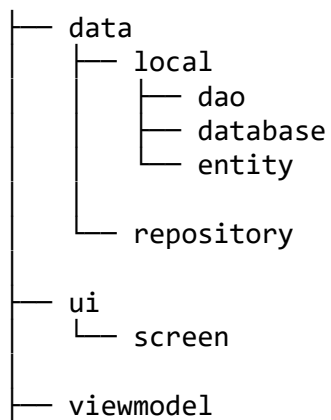
Kemudian klik:

Sync Now

## 6. Membuat Struktur Folder

Buat struktur folder berikut:

`com.example.loginmvvm`



## 7. Membuat Entity User

### Langkah 1

Klik kanan folder:

`entity`

Pilih:

New → Kotlin Class/File

Nama file:

User

## Langkah 2

Isi kode berikut:

```
package com.example.loginmvvm.data.local.entity

import androidx.room.Entity
import androidx.room.PrimaryKey

@Entity(tableName = "users")
data class User(

    @PrimaryKey(autoGenerate = true)
    val id: Int = 0,

    val username: String,

    val password: String
)
```

## 8. Membuat DAO

### Langkah 1

Buat file:

UserDao

Pada folder:

dao

### Langkah 2

Isi kode berikut:

```
package com.example.loginmvvm.data.local.dao

import androidx.room.Dao
import androidx.room.Insert
import androidx.room.Query
import com.example.loginmvvm.data.local.entity.User

@Dao
interface UserDao {

    @Insert
    suspend fun insert(user: User)
```

```

    @Query(
        "SELECT * FROM users WHERE username = :username AND password =
:password"
    )
    suspend fun login(
        username: String,
        password: String
    ): User?
}

```

## 9. Membuat Database Class

### Langkah 1

Buat file:

AppDatabase

Pada folder:

database

### Langkah 2

Isi kode berikut:

```

package com.example.loginmvvm.data.local.database

import android.content.Context
import androidx.room.Database
import androidx.room.Room
import androidx.room.RoomDatabase
import com.example.loginmvvm.data.local.dao.UserDao
import com.example.loginmvvm.data.local.entity.User

@Database(
    entities = [User::class],
    version = 1
)
abstract class AppDatabase : RoomDatabase() {

    abstract fun userDao(): UserDao

    companion object {

        @Volatile
        private var INSTANCE: AppDatabase? = null
    }
}

```

```

    fun getDatabase(context: Context): AppDatabase {

        return INSTANCE ?: synchronized(this) {

            val instance = Room.databaseBuilder(
                context.applicationContext,
                AppDatabase::class.java,
                "login_db"
            ).build()

            INSTANCE = instance

            instance
        }
    }
}

```

## 10. Membuat Repository

### Langkah 1

Buat file:

UserRepository

Pada folder:

repository

### Langkah 2

Isi kode berikut:

```

package com.example.loginmvvm.data.repository

import com.example.loginmvvm.data.local.dao.UserDao
import com.example.loginmvvm.data.local.entity.User

class UserRepository(
    private val dao: UserDao
) {

    suspend fun insert(user: User) {
        dao.insert(user)
    }
}

```

```

suspend fun login(
    username: String,
    password: String
): User? {
    return dao.login(username, password)
}
}

```

## 11. Membuat ViewModel

### Langkah 1

Buat file:

LoginViewModel

Pada folder:

viewmodel

### Langkah 2

Isi kode berikut:

```

package com.example.loginmvvm.viewmodel

import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.setValue
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.example.loginmvvm.data.local.entity.User
import com.example.loginmvvm.data.repository.UserRepository
import kotlinx.coroutines.launch

class LoginViewModel(
    private val repository: UserRepository
) : ViewModel() {

    var loginState by mutableStateOf("")
    private set

    fun login(
        username: String,
        password: String
    ) {

        viewModelScope.launch {

```

```

        val user = repository.login(
            username,
            password
        )

        loginState =
            if (user != null) {
                "Login Berhasil"
            } else {
                "Username atau Password Salah"
            }
    }
}

fun insertDummyUser() {
    viewModelScope.launch {
        repository.insert(
            User(
                username = "admin",
                password = "12345"
            )
        )
    }
}
}

```

## 12. Membuat ViewModel Factory

Buat file:

LoginViewModelFactory

Isi kode berikut:

```

package com.example.loginmvvm.viewmodel

import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelProvider
import com.example.loginmvvm.data.repository.UserRepository

class LoginViewModelFactory(
    private val repository: UserRepository
) : ViewModelProvider.Factory {

    override fun <T : ViewModel> create(

```



```

        modelClass: Class<T>
    ): T {

        return LoginViewModel(repository) as T
    }
}

```

## 13. Membuat UI Login Compose

### Langkah 1

Buat file:

LoginScreen

Pada folder:

screen

### Langkah 2

Isi kode berikut:

```

package com.example.loginmvvm.ui.screen

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.material3.Button
import androidx.compose.material3.Text
import androidx.compose.material3.TextField
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import com.example.loginmvvm.viewmodel.LoginViewModel

@Composable
fun LoginScreen(
    viewModel: LoginViewModel
) {

    var username by remember {

```

```

        mutableStateOf("")
    }

    var password by remember {
        mutableStateOf("")
    }

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp),

        verticalArrangement = Arrangement.Center
    ) {

        TextField(
            value = username,
            onChange = {
                username = it
            },
            label = {
                Text("Username")
            }
        )

        Spacer(modifier = Modifier.height(8.dp))

        TextField(
            value = password,
            onChange = {
                password = it
            },
            label = {
                Text("Password")
            }
        )

        Spacer(modifier = Modifier.height(16.dp))

        Button(
            onClick = {
                viewModel.login(
                    username,
                    password
                )
            }
        ) {

```

```

        Text("Login")
    }

    Spacer(modifier = Modifier.height(16.dp))

    Text(viewModel.loginState)
}
}

```

## 14. Menghubungkan ke MainActivity

Buka file:

MainActivity.kt

Ganti seluruh isi dengan:

```

package com.example.loginmvvm

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.lifecycle.viewmodel.compose.viewModel
import com.example.loginmvm.data.local.database.AppDatabase
import com.example.loginmvm.data.repository.UserRepository
import com.example.loginmvm.ui.screen.LoginScreen
import com.example.loginmvm.viewmodel.LoginViewModel
import com.example.loginmvm.viewmodel.LoginViewModelFactory

class MainActivity : ComponentActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {

        super.onCreate(savedInstanceState)

        val database =
            AppDatabase.getDatabase(this)

        val repository =
            UserRepository(
                database.userDao()
            )

        val factory =
            LoginViewModelFactory(repository)

        setContent {

```

```

        val viewModel: LoginViewModel =
            viewModel(factory = factory)

        androidx.compose.runtime.LaunchedEffect(Unit) {
            viewModel.insertDummyUser()
        }

        LoginScreen(viewModel)
    }
}

```

## 15. Menjalankan Aplikasi

### Langkah 1

Hubungkan emulator atau smartphone Android.

### Langkah 2

Klik tombol:

Run ►

## 16. Simulasi Login

### Data Dummy

| Username | Password |
|----------|----------|
| admin    | 12345    |

### Login Berhasil

Input:

Username : admin

Password : 12345

Output:

Login Berhasil

### Login Gagal

Input:

Username : admin  
Password : salah

Output:

Username atau Password Salah

## 17. Alur MVVM pada Aplikasi

User klik tombol login



Compose UI



ViewModel.login()



Repository.login()



DAO.login()



Room Database



SQLite



Data dikembalikan



ViewModel update state



Compose UI recompose

## Kesimpulan

Aplikasi login sederhana ini menunjukkan implementasi:

- MVVM Architecture
- Room Database
- SQLite
- Jetpack Compose
- Kotlin Coroutine

Arsitektur MVVM membantu:

- Memisahkan UI dan logic
- Membuat kode lebih rapi
- Mudah maintenance
- Mudah dikembangkan

Room membantu:

- Mengelola database lokal
- Membuat query lebih aman
- Mendukung asynchronous programming